

Apache Maven | POM,



Date _____
Page 10

Dependencies & Lifecycle

Normal Flow

JAR files

Demo.java → compiler → Demo.class
↓
JVM
↓
Output

Project Flow

Order.java
User.java
Payment.java

} Multiple WO's of files
in a project

Order.class
User.class
Payment.class

→ ZIP → not works

☐ Solution → JAR → Java Archives
Package

can contain 1) Multiple class files.

2) Resources (images/properties)

3) Folders/packages.

Use case → To share java code easily.

→ To use external libraries.

(mysql-connector.jar)



→ Making my Library →

Library
Demo.java
User.java

→ Non Executable Jar.
→ myLibrary.jar
(will not contain the libraries required for myLibrary)

mysql-connector.jar

(Libraries required for my library to run)

→ Making my Application →

Demo.java
User.java
Main.java
mysql-connector.jar

→ myApplication.jar
Executable Jar

→ My Code

Java Code + 3rd party Libraries (JAR)

↓
Compile

↓
Run → ClassPath

When we have to run a code from the JAR it first searches the code when it does not

Maven



Date _____
Page 12

☐ → MySQL + connector.jar file download.



Problems

1. Manual effort.
2. Versioning mismatch
3. Transitive dependencies.

☐ What is Maven?

→ Project Management Tool.

- ↳ 1. Folder structure
- ↳ 2. Compile code
- ↳ 3. Creates JAR code
- ↳ 4. Dependencies download (JAR)

pom.xml → full form → project object model.xml
pom.xml (Project Object Model)

⇒ super pom → parent of all pom.

effective pom → maven finally uses this.
(pom + super pom)

→ Maven uses 2 Repositories:

- ⇒ Maven Central
- ⇒ local .m2/Repository.
- ⇒ Other remote.



→ Complete Flow:

1. Maven read pom.xml
2. Local Repo check (m2/repository)
3. check Maven Central Repo
4. Store downloaded jar in local.

→ Life Cycle:

- 1. Clean life cycle (clean)
- 2. Default life cycle
 - validate → compile → test →
packages → verify → install →
deploy.
- 3. Site life cycle (site)